Learn Haskell by Building a Blog Generator

Source: Learn Haskell by Building a Blog Generator

Chapter 3 - Building an HTML printer library

- Code that is supposed to be part of some expression should be indented further than the beginning of that expression. When in doubt start a new line and tab over further than the line above you.
- A `newtype` declaration is a way to define a new, distinct type for an existing set of values. This is useful to reuse existing values but give them a different meaning and ensure we can't mix the two.

```
newtype <type-name> = <constructor> <existing-type>
newtype Html = Html String
```

- The first `Html` before the equal sign is a type and you will see it to the right of `::`. The second `Html` lives in the expressions namespace. These names can differ from each other but are often set to the same value. Both have to start with a capital letter.
- You use the constructor on the right to take a `String` argument and return something of the new type like `Html "world"`. Note that our newtype does not work for expressions that expect a `String`, so we lose concatenation via `<>` among other things when we make a `newtype`. We can however extract the `String` out of the expression.
- Using `newtype` has no runtime performance cost. The new type and the constructor are only there to help us distinguish between the type we created and the underlying type when we write our code and aren't needed when the code is running.

```
-- Pattern matching
case <expression> of
  <pattern> -> <expression>
  ...
  <pattern> -> <expression>

getStructureString :: Structure -> String
getStructureString struct =
  case struct of
    Structure str -> str

-- alternatively, we could use pattern matching for function arguments
func <pattern> = <expression>

getStructureString :: Structure -> String
getStructureString (Structure str) = str
```

- The composition operator feeds the output of one function as the input of another.

```
(.) :: (b -> c) -> (a -> b) -> a -> c
(.) f g x = f (g x)
```

- Lowercase values in type signatures are type variables. They can take any concrete value, but when the same variable occurs multiple times in the same signature, it must have a matching value. In the above, the `a` could be any type, but if the first `a` is `Int` then so is the second.
- We could have also used `type` but it is simply a type alias rather than a separate type. `type Html = String` makes `Html` a synonym for `String` and they can be used interchangeably. It adds clarity to our code but doesn't prevent you from mixing up an `Html` and `String` argument.
- We can define local names using `let`:

```
let
  <name> = <expression>
in
  <expression>
```

- You can expose internal modules of our libraries to provide some flexibility for advanced users. Internal modules are simply modules named `<something>.Internal` which export all of the functionality and implementation details in that module. We will write the implementation in `Html.Internal` which will export everything and import that module in `Html` and write an explicity export list as the public API. Internal modules are considered unstable and risky to use by convention but are a fairly common design pattern in Haskell.

Chapter 4 - Custom markup language

- We can use `data` to create customs types by grouping multiple types together and having alternative structures, something like a combination of structs and enums.

```
data <Type-name> <type-args>
  = <Data-constructor1> <types>
    <Data-constructor2> <types>
    ...

data Bool
  = True
  | False

data Person
  = Person String Int

-- Person could instead be a record
data Person
  = Person
    { name :: String
    , age :: Int
    }

data Tuple a b
  = Tuple a b

data Either a b
  = Left a
  | Right b
```

- In the `<types>` part, we can have as many types as we like unlike the restriction to one in `newtype`, and we can have alternative structures separated by `|` unlike `newtype` which has no alternatives.

- A *kind* is the "type" of a type. Kinda can either be `*`, which means saturated (or concrete) type, such as `Int` or `Person`, or else an `->` of two or more kinds which is a type of function taking kind(s) and returning a kind. Only types that have the kind `*` can have values.
- In Haskell, to emulate iteration with a mutable state we use recursion, calling the function again with the values we want the variables to have next iteration.
- In general, when trying to solve problems recursively, it is useful to think about the problem in three parts:
    1. Finding the *base case* (the most simple cases that we already know how to answer)
    2. Figuring out how to *reduce* the problem to something simpler (so it gets closer to the base case)
    3. *Mitigating the difference* between the reduced version and the solution we need to provide.

```haskell
myReplicate :: Int -> a -> [a]
myReplicate n x
  | n <= 0 = []
  | otherwise = x : replicate (n-1) x

even :: Int -> Bool
even 0 = True
even n = odd (n-1)

odd :: Int -> Bool
odd 0 = False
odd n = even (n-1)
```

- A function that does not return a result for some value (either by not terminating or throwing an error) is called a *partial function*. Writing partial functions is considered a bad practice. Make sure the functions you write return a result for every input, either by constraining the input using types or by encoding the absence of a result using types.
- The type class `Show` is for converting members to `String` for display.

```haskell
-- signature of the `show` function
show :: Show a => a -> String

-- a simplified version of the type class definition
class Show a where
  show :: a -> String

-- an implementation of `Show` for `Bool`
instance Show Bool where
  show x =
    case x of
      True -> "True"
      False -> "False"

-- or you can just derive it if your made up of `Show` members
data Structure
= Heading Natural String
| Paragraph String
| UnorderedList [String]
| OrderedList [String]
| CodeBlock [String]
deriving Show
```

- We can pattern match on constructors of a `data` declaration:

```haskell
import Data.Word (Word8)

data Color
  = RGB Word8 Word8 Word8

getBluePart :: Color -> Word8
getBluePart color =
  case color of
    RG _ _ blue -> blue

-- The List types special syntax works for pattern matching too
safeHead :: [a] -> Maybe a
safeHead list =
  case list of
    [] -> Nothing
    x : _ -> Just x

exactlyTwo :: [a] -> Maybe (a, a)
exactlyTwo list =
  case list of
    [x, y] -> Just (x, y)
    _ -> Nothing
-- x : y : [] is equivalent to [x, y]
```

- So we updated our `parseLines` function:

```
parseLines :: Maybe Structure -> [String] -> Document
parseLines context txts =
  case txts of
    [] -> maybeToList context
    currentLine : rest ->
      let
        line = trim currentLine
      in
        if line == ""
          then
            maybe id (:) context (parseLines Nothing rest)
          else
            case context of
              Just (Paragraph paragraph) ->
                parseLines (Just (Paragraph (unwords [paragraph, line])))
rest
              _ ->
                maybe id (:) context (parseLines (Just (Paragraph line))
rest)
```

- Consider the `maybe` lines in two parts. We have `maybe id (:) context` which prepends the context to the rest of the document followed by `parseLines Nothing rest` which is parsing the rest of the document. In the first part we want to prepend `context` to the rest of the document but cannot write `context : parseLines Nothing rest` because `context` has the type `Maybe Structure` and not `Structure`. If we do have a `Structure` to prepend, we prepend it. If not, we want to return the result of `parseLines Nothing rest` as is, hence `id`. `maybe` is a three argument function in this case returning one of two functions, `id` or `a:` for some `a` that was part of `Just a`, and then that function is applied to the result of the `parseLines` call.
- Consider the `maybe` line in two parts: `maybe id (:) context`, and what it's applied to, `parseLines Nothing rest` (the parse of everything after the blank line). We want to prepend `context` onto that parsed `rest`, but can't just write `context : parseLines Nothing rest`, because `context` is a `Maybe Structure`, not a `Structure`. `(:)` needs an actual `Structure` on its left. If we have one (`context` is `Just s`), we prepend it. If not (`context` is `Nothing`), we want the parsed `rest` back unchanged, hence `id`. So `maybe id (:) context` is a three-argument call to `maybe` that evaluates to one of two functions — `id` or (`s :`) for whatever `s` was inside `Just s` and that function is then applied to `parseLines Nothing rest` to produce the final list.

- It doesn't take much beyond understanding that to go from parsing paragraphs to handling our whole markup language.

```haskell
parseLines :: Maybe Structure -> [String] -> Document
parseLines context txts =
  case txts of
    -- done case
    [] -> maybeToList context

    -- Heading 1 case
    ('*' : ' ' : line) : rest ->
      maybe id (:) context (Heading 1 (trim line) : parseLines Nothing rest)

    -- Unordered list case
    ('-' : ' ' : line) : rest ->
      case context of
        Just (UnorderedList list) ->
          parseLines (Just (UnorderedList (list <> [trim line]))) rest

        _ ->
          maybe id (:) context (parseLines (Just (UnorderedList [trim line]))
rest)

    -- Ordered list case
    ('#' : ' ' : line) : rest ->
      case context of
        Just (OrderedList list) ->
          parseLines (Just (OrderedList (list <> [trim line]))) rest

        _ ->
          maybe id (:) context (parseLines (Just (OrderedList [trim line]))
rest)

    -- Code Block case
    ('>' : ' ' : line) : rest ->
      case context of
        Just (CodeBlock list) ->
          parseLines (Just (CodeBlock (list <> [trim line]))) rest

        _ ->
          maybe id (:) context (parseLines (Just (CodeBlock [trim line]))
rest)

    -- Paragraph case
    currentLine : rest ->
      let
        line = trim currentLine
      in
```

```
        if line == ""
          then
            maybe id (:) context (parseLines Nothing rest)
          else
            case context of
              Just (Paragraph paragraph) ->
                parseLines (Just (Paragraph (unwords [paragraph, line])))
rest
              _ ->
                maybe id (:) context (parseLines (Just (Paragraph line))
rest)
```

Chapter 5 - Gluing things together

- The dollar sign `($)` acts like invisible parenthesis around the expressions to the left and right of it so that `Html_ul_ $ map Html.p_ list` is treated like `(Html.ul_) (map Html.p_ list)`. It applies the function on the left to the argument on the right. It is right associative and very low precedence.
- A monoid is an instance of `Semigroup` so has `(<>)` but also has an identity.
- `Foldable` is a type class for kind `* -> *`.

```
fold :: (Foldable t, Monoid m) => t m -> m
foldMap :: (Foldable t, Monoid m) => (a -> m) -> t a -> m
-- consider a specialized `foldMap`
myFoldMap :: (Markup.Structure -> Html.Structure)
          -> [Markup.Structure]
          -> Html.Structure

instance Foldable [] where
  ...
  fold = List.mconcat
```

- `IO` forces a distinction from non-I/O expressions and will also require that in order to combine multiple `IO` operations, we will have to specify the order of the operations.

```haskell
-- some IO operations
putStrLn :: String -> IO ()
getLine :: IO String
getArgs :: IO [String]
lookupEnv :: String -> IO (Maybe String)
writefile :: FilePath -> String -> IO ()
```

- The meaning of `IO a` is that it is a description of a program (or subroutine) that, when execute, will produce some value of type `a`, and may do some I/O effects during execution. Evaluating an `IO` expression is pure because it will always reduce to the same description of a program, but executing it is impure. The Haskell runtime executes the entry point of the program (the `main` expression that must have type `IO ()`), and any other `IO` expression should be combined into the `main` expression to be executed.

```haskell
-- We are going to look at these operators for IO, but note they generalize
-- to other types too, and we will talk about those later.
(>>=) :: IO a -> (a -> IO b) -> IO b
-- Ex: Echo, (>>=) passes the result on the left to the operation on the
right
echo :: IO ()
echo = getLine >>= (\line -> putStrLn line)
-- Ex: Appending two inputs
getLine >>= \honorific ->
  getLine >>= \name ->
    putStrLn ("Hello " ++ honorific ++ " " ++ name)

-- (*>) is a more generalized version of (>>)
-- both run the first operation, discard the result, then run the second.
(*>) :: IO a -> IO b -> IO b
(>>) :: IO a -> IO b -> IO b
-- You can define these with (>>=)
a *> b = a >>= \_ -> b
helloWorld = putStrLn "hello" *> putStrLn "world"

-- pure is a more general version of return
-- It is useful when we want to do some uneffectful computation that depends
-- on IO
pure :: a -> IO a
-- Ex:
confirm :: IO Bool
confirm =
  putStrLn "Are you sure? (y/n)" *>
    getLine >>= \answer ->
      case answer of
        "y" -> pure True
        "n" -> pure False
        _ ->
          putStrLn "Invalid response. use y or n" *>
            confirm

-- fmap and <$>. (<$>) is the infix version of fmap.
fmap :: (a -> b) -> IO a -> IO b
excite = fmap (\line -> line ++ "!") getLine

-- Summary
-- chaining IO operations: passing the *result* of the left IO operation
-- as an argument to the function on the right.
-- Pronounced "bind".
(>>=) :: IO a -> (a -> IO b) -> IO b
```

```haskell
-- sequence two IO operations, discarding the payload of the first.
(*>) :: IO a -> IO b -> IO b

-- "lift" a value into IO context, does not add any I/O effects.
pure :: a -> IO a

-- "map" (or apply a function) over the payload value of an IO operation.
fmap :: (a -> b) -> IO a -> IO b
```

- Remember that IO is a description of a program and without combining it into `main` in some way it won't actually do anything.

```haskell
whenIO :: IO Bool -> IO () -> IO ()
whenIO cond action =
  cond >>= \result ->
    if result
      then action
      else pure ()

-- using whenIO with confirm presented above
main :: IO ()
main =
  putStrLn "This program will tell you a secret" *>
    whenIO confirm (putStrLn "IO is actually pretty awesome") *>
      putStrLn "Bye"
```

- The second `putStrLn` above is only execute if it is what `whenIO` returns.
- In Haskell, once we get into `IO`, there is no getting out. The only thing we can do is to build bigger IO computations by combining it with more IO computations. We like to keep `IO` minimal and push it to the edges of the program. This pattern is often called *Functional core, imperative shell*.
- We are going to need several more IO functions to write our program:

```haskell
import System.Directory (doesFileExist)
import System.Environment (getArgs)
-- The following are imported by Prelud
-- import System.IO (getContents, readFile, writeFile)

getArgs :: IO [String] -- Get the program arguments
getContents :: IO String -- Read all of teh content from stdin
readFile :: FilePath -> IO String -- Read all of the content from a file
writeFile :: FilePath -> String -> IO () -- Write a string into a file
doesFileExist :: FilePath -> IO Bool -- Checks whether a file exists
```

- Haskel provides *do notation* to chain `IO` actions, emulating imperative programming. A *do block* starts with the `do` keyword and continues with one or more "statements" which can be one of the following:

  1. An expression of type `IO ()`, such as:
  - `putStrLn "Hello"`
  - `if True then putStrLn "Yes" else putStrLn "No"`
  2. A `let` block, such as
  - `let x = 1`
  - or multiple let declarations

```haskell
let
  x = 1
  y = 2
-- Note that we do not write the `in` here.
```

  3. A binding `<variable> <- <expression>`, such as

```haskell
line <- getLine
```

- And the last "statement" must be an expression of type `IO <something>` and this will be the result type of the do block.

- These constructs are desugared (translated) by the Haskell compiler to:

  1. `<expression> *>`

2. `let ... in`
3. >>= <variable>

- The following two greetings are equivalent:

```haskell
greeting :: IO ()
greeting = do
  putStrLn "Tell me your name."
  let greet name = "Hello, " ++ name ++ "!"
  name <- getLine
  putStrLn (greet name)

greeting :: IO ()
greeting =
  putStrLn "Tell me your name." *>
    let
      greet name = "Hello, " ++ name ++ "!"
    in
      getLine >>= \name ->
        putStrLn (greet name)
```

- You can start a new project with cabal using `cabal init --libandexe` or with stack using `stack new`.

- Some important commands:

```
-- Putting commands that use "--" in quotes
cabal update      -- run to update information before fetching dependencies
cabal build       -- compile and also fetch and install dependencies
"cabal exec -- which hs-blog-gen" -- show exe location
"cabal run hs-blog-gen -- <program arguments>"    -- run the exe
cabal repl hs-blog  -- run ghci while loading dependencies and modules
cabal clean       -- deletes the build artifacts

stack build       -- compiles the project and install ghc and dependencies
"stack exec -- which hs-blog-gen" -- show exe location
"stack exec hs-blog-gen -- <program arguments>"   -- run the exe
stack ghci hs-blog    -- run ghci while loading dependencies and modules
stack clean       -- delete the build artifacts

-- also add this to .gitignore
dist
dist-newstype
.stack-work
```

- In this book, we are using **optparse-applicative** library for command-line argument parsing. We add it to the build-depends stanze in the executable section of our cabal file, since it would be needed by someone wishing to run the application but not necessarily by someone seeking to use the library.
- To use the library we model our options as an `Options` ADT and then build a `Parser a` that parses the command-line arguments an produces an `a`.

```haskell
inp :: Parser FilePath
inp =
  strOption
      ( long "input"
        <> short 'i'
        <> metavar "FILE"
        <> help "Input file"
      )

out :: Parser FilePath
out =
  strOption
    ( long "output"
      <> short 'o'
      <> metavar "FILE"
      <> help "Output file"
    )
```

- `strOption` is a parser builder. It takes a combined *option modifiers* as an argument, and returns a parser that will parse a string. As you see, we combine modifiers with `(<>)`, so the options are a `Semigroup`. Parsers are also Functors, and `fmap` lifts a function from `a -> b` to work as `Parser a -> Parser b`. Remember its infix version is `(<$>)`.

```haskell
class Functor f where
  fmap :: (a -> b) -> f a -> f b

-- 1. Identity law:
--    if we don't change the values, nothing should change
fmap id = id

-- 2. Composition law:
--    Composing the lifted functions is the same as composing them after fmap
fmap (f . g) == fmap f . fmap g

-- maybe is already a functor, but how would we legally implement fmap
mapMaybe :: (a -> b) -> Maybe a -> Maybe b
mapMaybe func maybeX =
  case maybeX of
      Nothing -> Nothing
      Just x -> Just (func x)

-- We have met several functors including IO, Maybe, [], and Parser
```

- We're going to extend our FilePath parser to work on our ADT, and also I need to list the definition of our ADT.

```haskell
data Options
  = ConvertSingle SingleInput SingleOutput
  | ConvertDir FilePath FilePath
  deriving Show

data SingleInput
  = Stdin
  | InputFile FilePath
  deriving Show

data SingleOutput
  = Stdout
  | OutputFile FilePath
  deriving Show

-- So we already have a parser for FilePath and SingleOutput is a function
-- from FilePath -> SingleInput, so let's use fmap to make it a parser from
-- Parser FilePath -> Parser SingleInput.
pInputFile :: Parser SingleInput
pInputFile = fmap InputFile parser
  where
    parser =
      strOption
        ( long "input"
          <> short 'i'
          <> metavar "FILE"
          <> help "Input File"
        )

pOutputFile :: Parser SingleOutput
pOutputFile = OutputFile <$> parser -- fmap and <$> are the same
  where
    parser =
      strOption
        ( long "output"
          <> short 'o'
          <> metavar "FILE"
          <> help "Output File"
        )
```

- Given a `Parser SingleInput` and a `Parser SingleOutput`, we want to combine them as a `Parser Options`. If we merely had `SingleInput` and `SingleOutput`, we could simply use the constructor of `ConvertSingle` since `ConvertSingle :: SingleINput -> SingleOutput -> Options`. It just so happens that `liftA2 :: (a -> b -> c) -> f a -> f b -> f c` which when made concrete can be `(SingleInput -> SingleOutput -> Options) -> (Parser SingleInput -> Parser SingleOutput -> Parser Options`.
- Beyond what a regular functor can do, which is to lift a function over a certain `f`, applicative functors allow us to apply a function to *multiple instances* of a certain `f`, as well as "lift" any value of type `a` into an `f a`.

```haskell
class Functor f => Applicative f where
  pure :: a -> f a
  liftA2 :: (a -> b -> c) -> f a -> f b -> f c
  (<*>) :: f (a -> b) -> f a -> f b

-- with liftA2
pConvertSingle :: Parser Options
pConvertSingle =
  liftA2 ConvertSingle pInputFile pOutputFile

-- with <$> and <*>
pConvertSingle' :: Parser Options
pConvertSingle' =
  ConvertSingle <$> pInputFile <*> pOutputFile

pInputDir :: Parser FilePath
pInputDir =
  strOption
    (long "input"
     <> short 'i'
     <> metavar "DIRECTORY"
     <> help "Input directory"
    )

pOutputDir :: Parser FilePath
pOutDir =
  strOption
    (long "output"
      <> short 'o'
      <> metavar "DIRECTORY"
      <> help "Output directory"
    )

pConvertDir :: Parser Options
pConvertDir =
  ConvertDir <$> pInputDir <*> pOutputDir
```

- We have yet to account that instead of using an input and output file, `ConvertSingle` could also potentially read `stdin` and output to `stdout`. We want to make our `--input` and `--output` flags optional and when they aren't specified use `stdin` / `stdout`. We can do this with `optional` from `Control.Applicative`.

```haskell
optional :: Alternative f => f a -> f (Maybe a)

class Applicative f => Alternative f where
  (<|>) :: f a -> f a -> f a
  empty :: f a
```

- `Alternative` is uncommon and mostly used for parsers. It allows us to combine two `Parser`s and if the first one fails to parse, try the other.

```haskell
pSingleInput :: Parser SingleInput
pSingleInput =
  fromMaybe Stdin <$> optional pInputFile

pSingleOutput :: Parser SingleOutput
pSingleOutput =
  fromMaybe Stdout <$> optional pOutputFile

pConvertSingle :: Parser Options
pConvertSingle =
  ConvertSingle <$> pSingleInput <*> pSingleOutput
```

- We also actually want two commands; `convert` for `ConvertSingle` and `convert-dir` for `ConvertDir`. This combination only handles `ConvertSingle`, but we can create a parser with commands with `subparser` and `command` functions:

```haskell
subparser :: Mod CommandFields a -> Parser a
-- command modifiers are constructed with `command` and are `Monoid`s
command :: String -> ParserInfo a -> Mod CommandFields a
-- a `ParserInfo a` can be constructed with info
info :: Parser a -> InfoMod a -> ParserInfo a

pConvertSingleInfo :: ParserInfo Options
pConvertSingleInfo =
  info
    (helper <*> pConvertSingle)
    (progDesc "Convert a single markup source to html")

pConvertSingleCommand :: Mod CommandFields Options
pConvertSingleCommand =
  command "convert" pConvertSingleInfo

pOptions :: Parser Options
pOptions =
  subparser
    (command
      "convert"
      (info
        (helper <*> pConvertSingle)
        (progDesc "Convert a single markup source to html")
      )
      <> command
      "convert-dir"
      (info
        (helper <*> pConvertDir)
        (progDesc "Convert a directory of markup files to html")
      )
    )

opts :: ParserInfo Options
opts =
  info (helper <*> pOptions)
    (fullDesc
      <> header "hs-blog-gen - a static blog generator"
      <> progDesc "Convert markup files or directories to html"
    )
```

Chapter 6 - Handling errors and multiple files

- A value of type `Either` is either a `Left a` or a `Right b`.

```haskell
data Either a b
  = Left a
  | Right b

Left True :: Either Bool b
Right 'a' :: Either a Char
```

- It is customary to use `Left` to indicate failure with some error value attached, and `Right` to represent success with the expected result. It often makes sense to describe the failure modes using an ADT.

```haskell
data ParseDigitError
  = NotADigit Char
  deriving Show

parseDigit :: Char -> Either ParseDigitError Int
parseDigit c =
  case c of
    '0' -> Right 0
    '1' -> Right 1
    '2' -> Right 2
    '3' -> Right 3
    '4' -> Right 4
    '5' -> Right 5
    '6' -> Right 6
    '7' -> Right 7
    '8' -> Right 8
    '9' -> Right 9
    _ -> Left (NotADigit c)

-- Either is both a Functor an Applicative.
max3chars :: Char -> Char -> Char -> Either ParseDigitError Int
max3chars x y z =
  (\a b c -> max a (max b c))
    <$> parseDigit x
    <*> parseDigit y
    <*> parseDigit z

-- The first Left is returned when using Applicative.
instance Applicative (Either e) where
  pure          = Right
  Left  e <*> _ = Left e
  Right f <*> r = fmap f r

-- The `Traversable` abstraction allows us to combine an unspecified
-- amount of values such as `Either ParseDigitError Int`.
ghci> map parseDigit "1234567"
[Right 1,Right 2,Right 3,Right 4,Right 5,Right 6,Right 7]
ghci> :t sequenceA
sequenceA :: (Traversable t, Applicative f) => t (f a) -> f (t a)
ghci> sequenceA (map parseDigit "1234567")
Right [1,2,3,4,5,6,7]
ghci> map parseDigit "1a2"
[Right 1,Left (NotADigit 'a'),Right 2]
ghci> sequenceA (map parseDigit "1a2")
Left (NotADigit 'a')
```

```
-- Doing `map` then `sequenceA` is a pattern called `traverse`:
ghci> :t traverse
traverse :: (Traversable t, Applicative f) => (a -> f b) -> t a -> f (t b)
ghci> traverse parseDigit "1234567"
Right [1,2,3,4,5,6,7]
ghci> traverse parseDigit "1a2"
Left (NotADigit 'a')
```

- We can use `traverse` on any two types where one implements `Applicative`, like `Either` or `IO`, and the other implements `Traversable`, like `[]` and `Map k`.

```
ghci> import qualified Data.Map as M -- from the containers package
ghci> file1 = ("output/file1.html", "input/file1.txt")
ghci> file2 = ("output/file2.html", "input/file2.txt")
ghci> file3 = ("output/file3.html", "input/file3.txt")
-- `fromList` takes a list of tuples
ghci> files = M.fromList [file1, file2, file3]
ghci> :t files :: M.Map FilePath FilePath -- FilePath is an alias of String
files :: M.Map FilePath FilePath :: M.Map FilePath

ghci> readFiles = traverse readFile
ghci> :t readFiles
readFiles :: Traversable t => t FilePath -> IO (t String)

ghci> readFiles files
fromList [("output/file1.html","I'm the content of file1.txt\n"),...]
ghci> :t readFiles files
readFiles files :: IO (Map String String)
```

- `Either` isn't actually a Functor or Applicative because its kind is `* -> * -> *` and not `* -> *`. However `Either e` has kind `* -> *` and can implement these type classes.

```
liftA2 :: Applicative => (a -> b -> c) -> f a -> f b -> f c
lifeA2 :: (a  -> b -> c) -> Either e a -> Either e b -> Either e c
```

- So we can combine two `Either` with the same type for the `Left` constructor.
- There are two ways to deal with combining functions that can return different errors:

    1. Make them return the same error type through an ADT with all possible errors.

2. Use a specialized error for each type but when they are composed together map the erro type of each to a more general erro type. This is normally done through `first` from the `Bifunctor` type class.

- Note that we normally want to use the output of one stage of parsing as the input to the next and thread the error through. This is what the `Monad` type class does best.

```haskell
tokenize :: String -> Either Error [Token]
parse :: [Token] -> Either Error AST
typecheck :: AST -> Either Error TypedAST

-- reminder of the type of fmap
fmap :: Functor f => (a -> b) -> f a -> f b
-- specialized for `Either Error`
fmap :: (a -> b) -> Either Error a -> Either Error b

-- here `a` is [Token] and `b` is `Either Error AST`:
> fmap parse (tokenize string) :: Either Error (Either Error AST)

-- We could handle this through pattern matching, but its cumbersome
case tokenize string of
  Left err ->
    Left err
  Right tokens ->
    case parse tokens of
      Left err ->
        Left err
      Right ast ->
        typecheck ast

-- instead we can flatten the nested Either structures
flatten :: Either e (Either e a) -> Either e a
flatten e =
  case e of
    Left l -> Left l
    Right x -> x

> flatten (fmap parse (tokenize string)) :: Either Error AST
> flatten (fmap typecheck (flatten (fmap parse (tokenize string))))
    :: Either Error TypedAST

-- So let's combine the recurring flatten/map pattern into a function:
flatMap :: (a -> Either e b) -> Either e a -> Either e b
flatMap func val = flatten (fmap func val)

> flatMap typecheck (flatMap parse (tokenize string)) :: Either Error
TypedAST
-- or use `flatMap` infix
> typecheck `flatMap` parse `flatMap` tokenize string
-- or define an operator (=<<) = flatMap
typeCheck =<< parse =<< tokenize string
```

- `flatten` and `flatMap` are called `join` and `=<<` (reverse bind) in Haskell.
- A type that can implement `Functor`, `Applicative` (most importantly `pure`), and the `join` function can implement an instance of the `Monad` type class.
- With functors, we were able to "lift" a function to work over the type implementing the functor type class:

```
fmap :: (a -> b) -> f a -> f b
```

- With applicative functors we were able to "lift" a function of multiple arguments over multiple values of a type implementing the applicative functor type class, and also lift a value into that type:

```
pure :: a -> f a
liftA2 :: (a -> b -> c) -> f a -> f b -> f c
```

- With monads we can nwo flatten (or "join" in Haskell terminology) types that implement the `Monad` interface:

```
join :: m (m a) -> m a
-- this is the =<< with the arguments reversed, pronounced "bind"
(>>=) :: m a -> (a -> m b) -> m b
```

- with `(>==)` we can write our compilation pipeline left-to-right:

```
> tokenize string >>= parse >>= typecheck
```

- The `Monad` interface means different things for different types. For `IO` this is ordering of effects, for `Either` it is early cutoff, for `Logic` it means backtracking computation, etc.
- `do` notation works for any type that is an instance of `Monad`.

```haskell
pipeline :: String -> Either Error TypedAst
pipeline string =
  tokenize string >>= \tokens ->
    parse tokens >>= \ast ->
      typecheck ast

-- with `do` notation
pipeline :: String -> Either Error TypedAst
pipeline string = do
  tokens <- tokenize string
  ast <- parse tokens
  typecheck ast

-- There is another way to do this: `>=>` and `<=<`:
(>=>) :: Monad m => (a -> m b) -> (b -> m c) -> a -> m c
(<=<) :: Monad m => (b -> m c) -> (a -> m b) -> a -> m c
-- compare with function composition:
(.) ::              (b ->   c) -> (a ->   b) -> a ->   c

pipeline = tokenize >=> parse >=> typecheck
pipeline = typecheck <=< parse <=< tokenize
```

- Using `Either` for error handling is useful because we encode possible errors using types and force users to acknowledge and handle them while being able to use the `Functor`, `Applicative`, and `Monad` interfaces to compose functions that might fail, reducing boilerplate and delaying the need to handle errors until it is appropriate.
- **Monad transformers** take a type that has an instance of monad as input and return a new type that implements the monad interface, stacking a new captability on top of it. Think if we wanted an `IO (Either Error a)` to handle all the ways `IO` can go wrong.

```haskell
newtype ExceptT e m a = ExceptT (m (Either e a))

-- `runExcept` converts an `ExceptT Error IO a` into a `IO (Either Error a)`
runExceptT :: ExceptT e m a -> m (Either e a)
```

-- `ExceptT` combined `Either` with whatever type of `Monad` is in `m`. `ExceptT e m` itself has a `Monad` instace.

```haskell
-- Generalized version
(>>=) :: Monad m => m a -> (a -> m b) -> m b
-- Specialized to ExceptT; the remaining m must be an instance of Monad
(>==) :: Monad m => ExceptT e m a -> (a -> ExceptT e m b) -> ExceptT e m b
-- Now consider bind for IO (Either Error a)
bindExceptT
  :: IO (Either Error a) -> (a -> IO (Either Error b)) -> IO (Either Error b)
bindExceptT mx f = do
x <- mx -- `x` has the type `Either Error a`
case x of
  Left err -> pure (Left err) -- lift our err to the monad
  Right y -> f y
-- Note how we didn't use any details of Error or IO other than the fact
-- IO is a Monad so supports do
bindExceptT
  :: Monad m => m (Either e a) -> (a -> m (Either e b)) -> m (Either e b)
bindExceptT mx f = do
x <- mx -- `x` has the type `Either e a`
case x of
  Left err -> pure (Left err)
  Right y -> f y
-- because `newtype ExceptT e m a = ExceptT (m (Either e a))`, we can unpack
-- the constructor and get:
bindExceptT :: Monad m => ExceptT e m a -> (a -> ExceptT e m b) -> ExceptT e m b
bindExceptT mx f = ExceptT $ do
-- `runExceptT mx` has the type `m (Either e a)`
-- `x` has the type `Either e a`
x <- runExceptT mx
case x of
  Left err -> pure (Left err)
  Right y -> runExceptT (f y)
```

- Note with stacking monad transformers, order matters. With `ExceptT Error IO a` we have an `IO` operation that when run will return `Either` an error or a value.

```haskell
throwError :: e -> ExceptT e m a  -- we can lift an error with `throwError`
-- or we can lift a monadic function to return a value of ExceptT instead
lift :: m a -> ExceptT e m a

getLine :: IO String
lift getLine :: ExceptT e IO String
-- techicall this is true, but we are specializing for concreteness
lift getLine :: MonadTrans t => t IO String
-- but getLine has no `Left` case, so this is really a function that always
-- succeeds! It just adds a wrapping with on real concept of failure.
lift = ExceptT . fmap Right

-- so if we had
readFile :: FilePath -> ExceptT IOError IO String
writeFile :: FilePath -> String -> ExceptT IOError IO ()
-- we could copose them
readFile "input.txt" >>= writeFile "output.html"
```

- The problem with the above is that `readFile` and `writeFile` have to have the same error type though. Should you have to deal with "disk space full" when reading? Instead of we give up on this approach for IO code and use Exceptions.
- When we stack `ExceptT` on top of `Identity` we get a type that is exactly `Either` called `Except`. You might prefer `Except` to `Either` given its more appropriate name and better API for error handling.
- The `Control.Exception` module provides us with the ability to `throw` exceptions from `IO` code, `catch` Haskell exceptions in `IO` code, and even convert them to `IO (Either ...)` with the function `try`.

```haskell
throwIO :: Exception e => e -> IO a

catch
  :: Exception e
  -> IO a         -- The computation to run
  -> (e -> IO a)  -- Handler to invoke if an exception is raised
  -> IO a

try :: Exception e => IO a -> IO (Either e a)
```

- By making a type an instance of the `Exception` type class, we can throw it and catch it in `IO` code.

```haskell
{-# language LambdaCase #-}

import Control.Exception
import System.IO

data MyException
  = ErrZero
  | ErrOdd Int
  deriving Show

instance Exception MyException

sayDiv2 :: Int -> IO ()
sayDiv2 n
  | n == 0 = throwIO ErrZero
  | n `mod` 2 /= 0 = throwIO (ErrOdd n)
  | otherwise = print (n `div` 2)

main :: IO ()
main =
  catch
    ( do
      putStrLn "Going to print a number now."
      sayDiv2 7
      putStrLn "Di you like it?"
    )
    ( \case
      ErrZero ->
        hPutStrLn stderr "Error: We don't support dividing zeroes for some
reason"
      ErrOdd ->
        hPutStrLn stderr ("Error: " <> show n <> " is odd and cannot be
divided by 2")
    )
```

- `LambdaCase` let's us do the above instead of `\e -> case e of`.
- `catches` works like `catch` but takes a list of Handlers.
- Catching `SomeException` will catch any exception.

- `bracket` and `finally` help us handle resource acquisition more safely when errors are present. `bracket` makes sure we always close a handle afterwards.

```haskell
import Control.Exception (bracket)

main :: IO ()
main = do
-- ...
  ConvertSingle input output ->
      let
        -- Here, action is the next steps we want to do.
        -- It takes as input the values we produce,
        -- uses it, and then returns control for us to clean-up
        -- afterwards.
        withInputHandle :: (String -> Handle -> IO a) -> IO a
        withInputHandle action =
          case input of
            Stdin ->
              action "" stdin
            InputFile file ->
              bracket
                (OpenFile file ReadMode)
                hClose
                (action file)

        -- Note that in both functions our action can return any `a`
        -- it wants.
        withOutputHandle :: (Handle -> IO a) -> IO a
        withOutputHandle action =
          case output of
            Stdout ->
              action stdout
            OutputFile file -> do
              exists <- doesFileExist file
              shouldOpenFile <-
                if exists
                  then confirm
                  else pure True
              if shouldOpenFile
                then
                  bracket (openFile file WriteMode) hClose actoin
                else
                  exitFailure
      in
        withInputHandle (\title -> withOutputHandle . HsBlog.convertSingle
title)
```

- There's a custom function that does a similar thing to `bracket (openFile file <mode>) hClose` called `withFile`.
- The Haskell language designers have designed `IO` to use exceptions instead of `Either`, so they are a good option for effectful computations. However `Either` is more appropriate for uneffectful code.
- `applyIoOnList` tries to apply an `IO` function ona list of values and document successes and failures. For each thing, it returns the thing itself along with the result of applying the `IO` function as an `Either` and the `Left` side is a `String` representation of the error.

```
applyIoOnList :: (a -> IO b) -> [a] -> IO [(a, Either String b)]
applyIoOnList action inputs = do
  for inputs $ \input -> do
    maybeResult <-
      catch
        (Right <$> action input)
        ( \(SomeExceptino e) -> do
          pure $ Left (displayException e)
        )
    pure (input, maybeResult)
```

Chapter 7 - Passing environment variables

- `ReaderT` is another *monad transformer* with instances of `Functor`, `Applicative`, `Monad`, and `MonadTrans`

```
newtype ReaderT r m a = ReaderT (r -> m a)
type Reader r a = ReaderT r Identity a
```

- The `r` usually represents the environment we want to share and `m a` is the underlying result. Usually it is `IO` or `Identity` depending upon whether we want to share an environment between effectful or uneffectful computations. `ReaderT` carries a value of type `r` and passes it around to other functions when we use `Applicative` or `Monad` interfaces instead of us needing to pass it around manually. We can use `ask` to access `r`. For us, this means instead of passing around an `Env`, we can convert our functions to use `ReaderT`, uneffectful ones returning `ReaderT Env Identity a` instead of `a` and effectful ones returning `ReaderT Env IO a` instead of `IO a.` Remember that `Applicative` and `Monad` expect the type that implements their interfaces to have the kind `* -> *`. This

means that `ReaderT r m` implements these interfaces so the `f` or `m` in the type signature of `<*>` or `>>=` is of type `ReaderT r m`. It also means that when we compose functions, they must share the same `r` and `m` for environment type and `m` type, much as our composed `Either e` needed to share errors `e`.

```
-- recall, liftA2 lifts a function to work across Applicatives
liftA2 :: Applicative f => (a -> b -> c) -> f a -> f b -> f c
-- Specialize: replace `f` with `ReaderT Env IO`
liftA2 ::
  (a -> b -> c) -> ReaderT Env IO a -> ReaderT Env IO b -> ReaderT Env IO c
-- now with the newtype for ReaderT unpacked
liftA2 :: (a -> b -> c) -> (Env -> IO a) -> (Env -> IO b) -> (Env -> IO c)
-- Now how would we implement this specailized liftA2
specialLiftA2 ::
  (a -> b -> c) -> (Env -> IO a) -> (Env -> IO b) -> (Env -> IO c)
specialLiftA2 combine funcA funcB env =
  liftA2 combine (funcA env) (funcB env)
-- clearly when we give `specialLiftA2 and `env`, it returns type `IO c`
-- but when partially applied, it returns an `Env -> IO c`
```

- How to use a `ReaderT`:

```haskell
-- Instead of defining a function like this:
txtsToRenderedHtml :: Env -> [(FilePath, String)] -> [(FilePath, String)]
-- We define it like this:
txtsToRenderedHtml :: [(FilePath, String)] -> Reader Env [(FilePath, String)]
-- But before ReaderT our function was (we manually threaded env):
txtsToRenderedHtml :: Env -> [(FilePath, String)] -> [(FilePath, String)]
txtsToRenderedHtml env txtFiles =
  let
    txtOutputFiles = map toOutputMarkupFile txtFiles
    index = ("index.html", buildIndex env txtOutputFiles)
    htmlPages = map (convertFile env) txtOutputFiles
  in
    map (fmap Html.render) (index : htmlPages)
-- Now we compose the relevant functions and use do notation:
txtsToRenderedHtml :: [(FilePath, String)] -> Reader Env [(FilePath, String)]
txtsToRenderedHtml txtFiles = do
  let
    txtOutputFiles = map toOutputMarkupFile txtFiles
  index <- (,) "index.html" <$> buildIndex txtOutputFiles
  htmlPages <- traverse convertFile txtOutputFiles
  pure $ map (fmap Html.render) (index : htmlPages)
```

- To use our `Env`, we need to *extrac* it from the `Reader` with `ask`.

```haskell
ask :: ReaderT r m r
-- Before our `env` was passed in.
convertFile :: Env -> (FilePath, Markup.Document) -> (FilePath, Html.Html)
convertFile env (file, doc) =
  (file, convert env (takeBaseName file) doc)
-- `convert` still takes `env` as an argument
convertFile :: (FilePath, Markup.Document) -> Reader Env (FilePath,
Html.Html)
convertFile (file, doc) = do
  env <- ask
  pure (file, convert env (takeBaseName file) doc)
```

- Part of the value of transformers is with one `<-` we get the underlying value through multiple layers of wrapping, since each transformers delegates downward to the underlying bind.
- We use `runReader` or `runReaderT` when we are ready to supply the environment.

```haskell
runReader :: Reader r a -> (r -> a)
runReaderT :: ReaderT r m a -> (r -> m a)

convertDirectory :: Env -> FilePath -> FilePath -> IO ()
convertDirectory env inputDir outputDir = do
  DirContents filesToProcess filesToCopy <- getDirFilesAndContent inputDir
  createOutputDirectoryOrExit outputDir
  let
    outputHtmls = runReader (txtsToRenderedHtml filesToProcess) env
  copyFiles outputDir filesToCopy
  writeFiles outputDir outputHtmls
  putStrLn "Done."
```

- Sometimes we use functions that don't need the entire `Env` so we transform it before passing in the result.

```haskell
outer :: Reader BigEnv MyResult
outer = do
  env <- ask
  pure (inner (extractSmallEnv env))

inner :: SmallEnv -> MyResult
inner = undefined

extractSmallEnv :: BigEnv -> SmallEnv
extractSmallEnv = undefined

-- But if `inner` uses a `Reader SmallEnv` we can use `runReader` to
-- convert `inner` to a normal function.
outer :: Reader BigEnv MyResult
outer = do
  env <- ask
  -- Here the type of `runReader inner` is `SmallEnv -> MyResult`
  pure (runReader inner (extractSmallEnv env))

inner :: Reader SmallEnv MyResult
inner = undefined

extractSmallEnv :: BigEnv -> SmallEnv
extractSmallEnv = undefined

-- This patter is captured by `withReaderT` which also works for `Reader`
withReaderT :: (env2 -> env1) -> ReaderT env1 m a -> ReaderT env2 m a

outer :: Reader BigEnv MyResult
outer = withReaderT extractSmallEnv inner
```

Chapter 8 - Writing tests

- We will be using hspec to test our project.
- `hspec-discover` will find all the `Spec` files and run all the tests it discovered. It requires `{-# OPTIONS_GHC -F -pgmF hspec-discover #-}` in the "main" file of the test suite which for us is `test/Spec.hs`.

```haskell
{-# language QuasiQuotes #-}  -- required for multiline strings

import Text.RawString.QQ      -- required for multiline string

-- We could define tests directly in spec, but here we call simple
spec :: Spec
spec = do
  describe "Markup parsing tests" $ do
    simple
    multiline

simple :: Spec
simple = do
  describe "simple" $ do
    it "empty" $
      shouldBe
        (parse "")
        []

    it "paragraph" $
      shouldBe
        (parse "hello world")
        [Paragraph "hello world"]

multiline :: Spec
multiline = do
  describe "Multi-line tests" $ do
    it "example3" $
      shouldBe
        (parse example3)
        example3Result

example3 :: String
example3 = [r|
Remember that multiple liens with no separation
are grouped together into a single paragraph
but list items remain separate.

# Item 1 of a list
# Item 2 of the same list
|]

example3Result :: Document
example3Result =
  [ Paragraph "Remember that multiple lines with no separation are group
```

```
together into a single paragraph but list items remain separate."
  , OrderedList
    [ "Item 1 of a list"
    , "Item 2 of the same list"
    ]
  ]
```

- For `hspec-discover` to find tests, their modules name must end with `Spec` and they must define a value `spec :: Spec` (which describes the test) and export it outside of the module (for example, in the export list).
- We use `describe` to describe a group of tests, `it` to add a new test, and `shouldBe` to compare two values using their `Eq` instance.
- `hspec-discover` saves us having to write a `Main.hs` that lists all our tests, but each test file does need to be added to the other-modules stanza for our test-suite. `hpack` is an option to auto-scan directories.
- For multline strings and to avoid escaping, we are using `raw-strings-qq`. It requires `{-# language QuasiQuotes #-}` and `import Text.RawString.QQ` in the test file. Examples in code above.

Chapter 9 - Generating documentation

- Run haddock with `cabal haddock` or `stack haddock`.
- Haddock works with both single line `(--)` and multiline `{- -}` comments. You either prefix the comment block with `|` before the definition or `^` after the definition.

```haskell
-- | Construct an HTML page from a `Head`
--   and a `Structure`.
html_
  :: Head -- ^ Represents the @\<head\>@ section in an HTML file
  -> Structure -- ^ Represents the @\<body\>@ section in an HTML file
  -> Html
html_ = ...


{- | Represents a single markup structure. Such as:

- A paragraph
- An unordered list
- A code block
-}
data Structure
  = Heading Natural String
  -- ^ A section heading with a level
  | Paragraph String
  -- ^ A paragraph
  | UnorderedList [String]
  -- ^ An unordered list of strings
  | OrderedList [String]
  -- ^ An ordered list of strings
  | CodeBlock [String]
  -- ^ A code block
  deriving (Eq, Show)


{- | Markup to Html conversion module.

This module handles converting documents written in our custom
Markup language into HTML pages.
-}
module HsBlog.Convert where
```

- Annotate the modules, types, and the top-level definitions which are exported from your project with some high-level description of what they are used for.
- We can separate our module into sections by adding heading prefixed with a number of `*`.

```haskell
-- * HTML EDSL

html_ :: Head -> Structure -> Html
html_ = ...

-- ** Structure

p_ :: Content -> Structure
p_ = ...

...

-- ** Content

txt_ :: String -> Content
txt_ = ...

link_ :: FilePath -> Content -> Content
link_ = ...
```

-- You can also add heading to the export list:

```
module HsBlog.Html
  ( -- * HTML EDSL
    Html
  , html_

    -- ** Combinators used to construct the @\<head\>@ section
  , Head
  , Title
  , title_
  , stylesheet_
  , meta_

    -- ** Combinators used to construct the @\<body\>@ section
  , p_
  , h_
  , ul_
  , ol_
  , code_

    -- ** Render HTML to String
  , render

    -- ** Structure
  , Structure

    -- ** Content
  , Content
  , txt_
  , img_
  , link_
  , b_
  , i_
  )
where
```

- Haddock supports formatting in the content of our comments.
  - Hyperlink identifiers by surrounding them with `: `Heading`
  - Write `monospace text` by surrounding it with @: @Paragraph "Hello"@
  - Add *emphasis* to text by surrounding it with /: /this is emphasised/
  - Add **bold** to text by surrounding it with __: **this is bold**

Chapter 10 - Recap

In this book, we've implemented a very simple static blod generator while learning Haskell as we go.

- We've learned about basic Haskell building blocks, such as definitions, functions, types, modules, recursion, pattern matching, type classes, IO, and exceptions.
- We've learned about EDSLs and used the *combinator pattern* to implement a composable html generation library.
- We've learned how to leverage types, modules, and smart constructors to make invalid states unrepresentable.
- We've learned how to represent complex data using ADTs.
- We've learned how to use pattern matching to transform ADTs, and how to use recursion to solve problems.
- We've used the *functional core, imperative shell* approach to build a program that handles IO and applies our domain logic to user inputs.
- We've learned about abstractoins such as monoid, functors, and monads, and how they can help us reuse code and convey information about shared interfaces.
- We've learned how to create fancy command-line interfaces, write tests, and generate documentation.